

GM UNIVERSITY

Davangere, Karnataka

Faculty of Computing & IT

Master of Computer Applications · First Semester

ADVANCED JAVA PROGRAMMING

Assignment-1 Report

Design and Development of a Campus Lost-and-Found Management Portal Using Advanced Java

Java Servlets · JSP · JavaBeans · JDBC · MySQL · MVC

SUBMITTED BY

Aniket C Tegginamath

USN: [USN Number]

MCA, First Semester

UNDER THE GUIDANCE OF

Mr. Rajashekhara G C

Faculty of Computing & IT

GM University, Davangere

Table of Contents

1. Introduction 4
1.1 Background 4
1.2 Purpose of the Project 4
1.3 Scope of the Project 4
1.4 Organisation of the Report 4
2. Problem Definition and Analysis 5
2.1 Problem Statement 5
2.2 Existing System 5
2.3 Proposed System 5
2.4 Feasibility Study 6
3. Objectives 7
4. Requirement Specification 8
4.1 Technologies Used 8
4.2 Hardware Requirements 8
4.3 Software Requirements 8
4.4 Functional Requirements 9
4.5 Non-Functional Requirements 10
5. System Design 11
5.1 System Architecture (MVC) 11
5.2 Context Diagram (DFD Level 0) 11
5.3 Use Case Diagram 12
5.4 Package and Class Design 12
5.5 URL Mapping 13
5.6 Claim and Recovery Workflow 14
5.7 Project Structure 14
6. Database Design 15
6.1 Database 15
6.2 Table 1: users 15
6.3 Table 2: items 15
6.4 Table 3: claims 15
6.5 Entity-Relationship Diagram 16
6.6 Database Creation SQL Script 16
6.7 Status Values 17
7. Implementation 18
7.1 HTML Forms and HTTP 18
7.2 Java Servlets 19
7.3 JSP 20
7.4 JavaBeans and MVC Integration 21
7.5 JDBC 22
7.6 Session Management 23
7.7 Security Measures 24

7.8 User Interface Design (Version 2) 24
7.9 Server-Side Validation and Business Rules 25
8. Application Screens 26
9. Testing 35
9.1 Testing Approach 35
9.2 Test Cases and Results 35
9.3 Database Verification 37
10. Deployment 38
10.1 Deployment Environment 38
10.2 Steps to Deploy in Eclipse 38
10.3 Deploying as a WAR File 38
10.4 Default Accounts 39
11. Results and Expected Output 40
11.1 Expected Output Achieved 40
11.2 Course Outcome Mapping 40
12. Conclusion and Future Enhancements 41
12.1 Conclusion 41
12.2 Future Enhancements 41
References 42

1. Introduction

1.1 Background

A university campus is a busy place. Hundreds of students and staff move between classrooms, laboratories, the library, the canteen and the hostels every day. In this movement, small but important belongings are easily lost: an identity card falls out of a pocket, a calculator is left in an exam hall, a phone is forgotten in a seminar hall. At the same time, many honest people find such items and want to return them, but they do not know who the owner is.

Most campuses manage this through notice boards, announcements or WhatsApp groups. These methods are slow, are not searchable and keep no record. A web portal that everyone on campus can open from a browser solves this problem in a simple and organised way.

1.2 Purpose of the Project

The purpose of this assignment is to design and develop a **Campus Lost-and-Found Management Portal** using the technologies taught in the Advanced Java Programming course: HTML forms and HTTP, Java Servlets, JSP, JavaBeans, JDBC and session management. The project shows how these technologies work together in a real, database-driven web application that follows the MVC architecture.

1.3 Scope of the Project

The application covers the complete life of a lost or found item on campus:

- registration and login of users and the administrator;
- reporting of lost items and found items;
- keyword and category based search with item details;
- ownership claims with proof, and verification by the administrator;
- status tracking from **Open** to **Claim Pending** to **Closed** (recovered);
- administrator tools to filter, update, remove and summarise reports.

1.4 Organisation of the Report

Chapter 2 analyses the problem. Chapter 3 lists the objectives. Chapter 4 gives the requirement specification. Chapter 5 explains the system design and Chapter 6 the database design. Chapter 7 describes the implementation of each Advanced Java technology. Chapter 8 shows the application screens. Chapter 9 covers testing and Chapter 10 explains deployment. Chapter 11 maps the results to the expected output, and Chapter 12 concludes the report.

2. Problem Definition and Analysis

2.1 Problem Statement

Students, faculty members and visitors may occasionally lose personal belongings such as identity cards, books, laptops, mobile phones, wallets, keys and other items within the university campus. Similarly, people may find belongings and wish to return them to their owners.

The task is to develop a Campus Lost-and-Found Management Portal using Advanced Java technologies. The application should allow users to report lost items, submit details of found items, search for reported belongings and submit ownership claims. An administrator should be able to manage reports, verify claims and update the recovery status. The application must integrate HTML forms, Java Servlets, JSP, JavaBeans, JDBC and a relational database, and must follow the Model-View-Controller (MVC) architecture.

2.2 Existing System

Method used today	Problem
Notice board near the office	Few people read it; notes get lost or removed
Classroom announcements	Reach only one class; no record is kept
WhatsApp / social media groups	Messages get buried; not searchable; privacy issues
Item kept at the security desk	Owner does not know it is there; no proof of ownership is checked

Drawbacks of the existing system: there is no central record, no search, no proof of ownership, no status tracking and no accountability for handing over an item.

2.3 Proposed System

The proposed portal keeps every lost and found report in one MySQL database. Any registered user can search it from a browser. A person who believes an item is theirs submits a claim with proof, and the administrator verifies the claim before the item is handed over. Every step is recorded with dates, status and remarks.

Advantages of the proposed system:

- One searchable place for all lost and found items on campus.
- Ownership is verified by the administrator using the details given in the claim.

- Clear status for every item (Open, Claim Pending, Closed) and every claim (Pending, Approved, Rejected).
- Users can follow their own reports and claims at any time.
- The administrator gets a live summary of reported and recovered items.
- Secure login with hashed passwords and role-based access.

2.4 Feasibility Study

Type	Analysis
Technical	All tools (JDK, Tomcat, MySQL, Eclipse) are free and already used in the MCA lab. The team knows Java.
Operational	Users only need a browser. The screens are simple forms that need no training.
Economic	No licence cost. It runs on an existing college server or a lab computer.
Schedule	The modules are small and can be finished within the assignment period.

3. Objectives

The objectives of the assignment and how each one is achieved in the project are shown below.

#	Objective	Achieved through
1	Design HTML forms for reporting lost and found items	Registration, login, report, search and claim forms with client-side validation
2	Understand HTTP request and response mechanisms	GET for reading data, POST for changing data, redirects, status codes 403 and 404
3	Develop Java Servlets to process user requests	Ten servlets, one for each main operation
4	Use JSP to display item details dynamically	Item lists, details, claims and dashboard built from request attributes
5	Implement JavaBeans and MVC architecture	User, Item and Claim beans; Servlet, JSP and DAO layers kept separate
6	Establish database connectivity using JDBC	DBConnection class and three DAO classes
7	Implement session management and user authentication	HttpSession, cookie, URL rewriting, role check and logout
8	Develop and deploy a database-driven web application	WAR file deployed and tested on Apache Tomcat 10.1

4. Requirement Specification

4.1 Technologies Used

Layer	Technology
Front End	HTML, CSS and JavaScript
Server-Side Programming	Java Servlets and JSP
Business Logic	JavaBeans
Database Connectivity	JDBC (MySQL Connector/J 8.4 — Type 4 driver)
Database	MySQL 8.x
Web Server	Apache Tomcat 10.1
Development Environment	Eclipse IDE for Enterprise Java and Web Developers
Language	Java 17

4.2 Hardware Requirements

Component	Minimum
Processor	Intel Core i3 or equivalent
RAM	4 GB (8 GB recommended for Eclipse)
Hard disk	2 GB free space
Network	LAN / Wi-Fi for campus access

4.3 Software Requirements

Software	Version
Operating system	Windows 10/11, Linux or macOS
JDK	17 or later
Apache Tomcat	10.1.x
MySQL Server	8.x
IDE	Eclipse 2023 or later (Enterprise Java edition)
Browser	Chrome, Edge or Firefox

4.4 Functional Requirements

A. User Module

1. Provide user registration and login facilities.
2. Allow users to report lost items.
3. Allow users to report items they have found.
4. Enter item name, category, description, location and date.
5. Search for lost or found items using keywords and categories.
6. View the details of reported items.
7. Submit a claim for an item believed to belong to them.
8. View the status of submitted claims.
9. View their own lost and found reports.
10. Logout securely.

B. Item Management Module

1. Maintain records of all lost and found items.
2. Categorise items such as electronics, books, identity cards, accessories and personal belongings.
3. Maintain the date and location where an item was lost or found.
4. Update the status of an item.
5. Display recently reported items.
6. Mark successfully recovered items as closed.

C. Ownership Claim Module

1. Allow users to submit ownership claims.
2. Capture relevant information to help establish ownership.
3. Allow administrators to review submitted claims.
4. Approve or reject claims.
5. Maintain claim status and verification remarks.

D. Administrator Module

1. Provide administrator login.
2. View all lost and found reports.

3. Search and filter reported items.
4. Review ownership claims.
5. Approve or reject claims.
6. Update item recovery status.
7. Remove inappropriate or duplicate reports.
8. View summary information about reported and recovered items.

4.5 Non-Functional Requirements

Requirement	Description
Security	SHA-256 password hashing, PreparedStatement against SQL injection, HTML escaping against script injection, role-based access, session timeout
Reliability	Claim operations use JDBC transactions with commit and rollback
Usability	Simple menus, clear messages, responsive layout
Maintainability	MVC packages and views stored inside WEB-INF
Portability	Runs on any OS that supports Java, Tomcat and MySQL

5. System Design

5.1 System Architecture (MVC)

The application follows the Model-View-Controller architecture. The **Controller** (Servlets) receives every request, the **Model** (JavaBeans and DAO classes) holds data and talks to the database, and the **View** (JSP pages) produces the HTML. Because the JSP views are placed inside `WEB-INF/views`, a browser cannot open them directly; every page must pass through a servlet, which checks the session first.

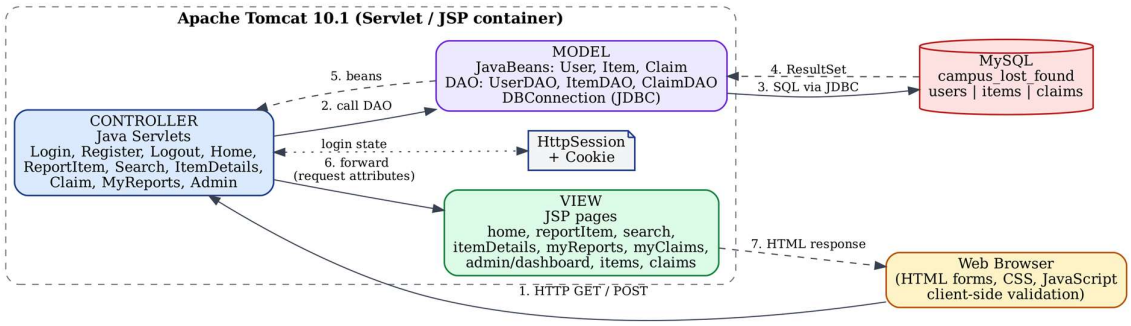


Figure 5.1: MVC architecture of the Campus Lost-and-Found Portal

Request-response flow:

1. The browser sends an HTTP GET or POST request to a servlet.
2. The servlet checks the session and reads the form parameters.
3. It calls a DAO method, which runs SQL through JDBC.
4. The DAO converts the ResultSet into JavaBeans.
5. The servlet stores the beans as request attributes.
6. The servlet forwards the request to a JSP page.
7. The JSP reads the beans and returns the HTML response.

5.2 Context Diagram (DFD Level 0)

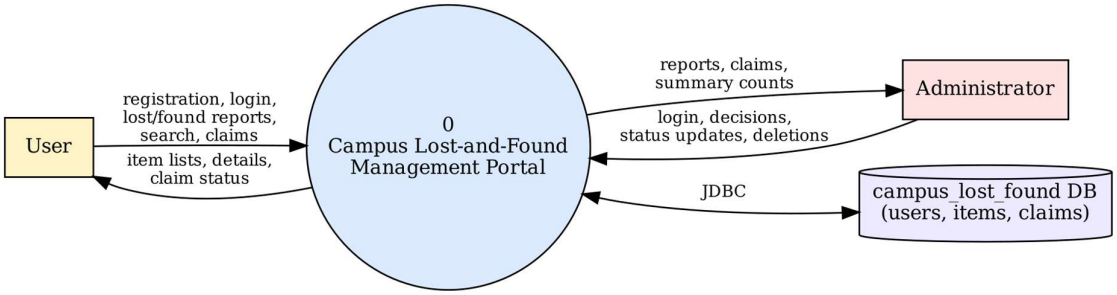


Figure 5.2: Context-level data flow diagram

5.3 Use Case Diagram

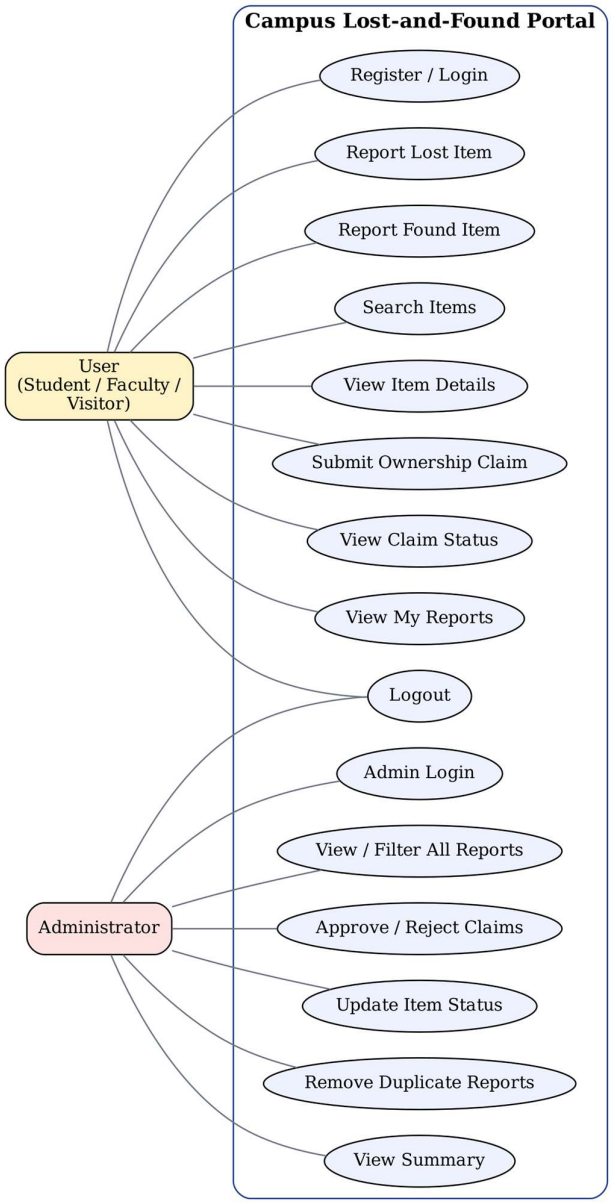


Figure 5.3: Use cases for the User and the Administrator

5.4 Package and Class Design

Package	Classes	Role in MVC
`com.campus.lostfound.bean`	User, Item, Claim	Model (data)
`com.campus.lostfound.dao`	DBConnection, UserDAO, ItemDAO, ClaimDAO	Model (database access)

Package	Classes	Role in MVC
`com.campus.lostfound.controller`	RegisterServlet, LoginServlet, LogoutServlet, HomeServlet, ReportItemServlet, SearchServlet, ItemDetailsServlet, ClaimServlet, MyReportsServlet, AdminServlet	Controller
`com.campus.lostfound.util`	PasswordUtil, SessionUtil, HtmlUtil	Helpers
`webapp` and `WEB-INF/views`	login.jsp, home.jsp, reportItem.jsp, search.jsp, itemDetails.jsp, myReports.jsp, myClaims.jsp, admin/*.jsp	View

5.5 URL Mapping

URL	Method	Servlet	Purpose
`/register`	POST	RegisterServlet	Create account
`/login`	GET / POST	LoginServlet	Show form / authenticate
`/home`	GET	HomeServlet	Recently reported items
`/report`	GET / POST	ReportItemServlet	Lost or found report form / save
`/search`	GET	SearchServlet	Keyword and category search
`/item`	GET	ItemDetailsServlet	Item details and claim form
`/claims`	GET / POST	ClaimServlet	My claims / submit claim
`/myReports`	GET	MyReportsServlet	My lost and found reports
`/admin`	GET / POST	AdminServlet	Dashboard, reports, claims, decisions
`/logout`	GET	LogoutServlet	End session

5.6 Claim and Recovery Workflow

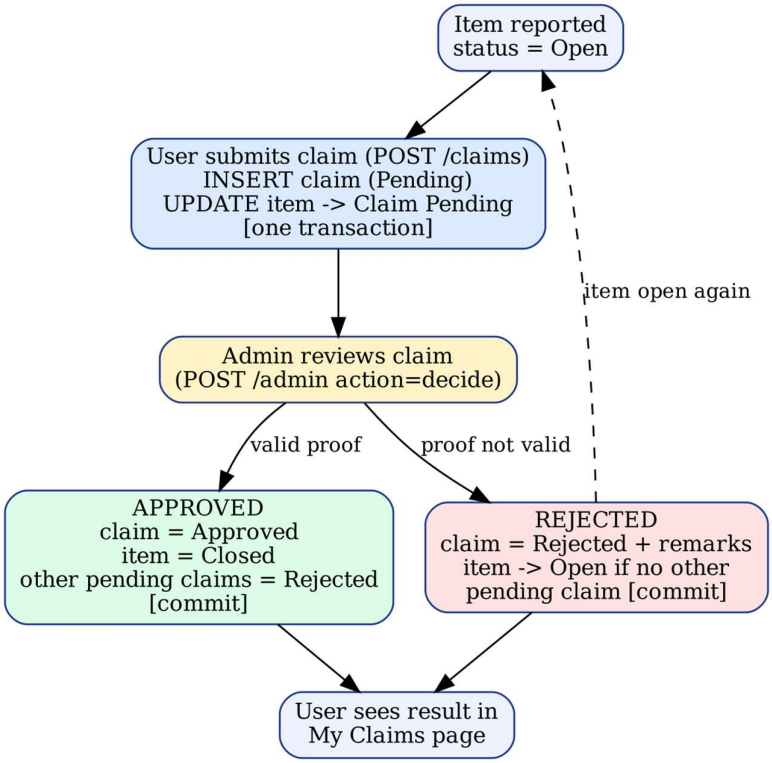


Figure 5.4: Item status and claim verification workflow

5.7 Project Structure

```
CampusLostFound/
|-- database/campus_lost_found.sql
`-- src/main/
    |-- java/com/campus/lostfound/
    |   |-- bean/        User, Item, Claim
    |   |-- dao/         DBConnection, UserDAO, ItemDAO, ClaimDAO
    |   |-- controller/  10 Servlets
    |   `-- util/        PasswordUtil, SessionUtil, HtmlUtil
    `-- webapp/
        |-- index.jsp  login.jsp  register.html  error.jsp
        |-- includes/  header.jsp  footer.jsp
        |-- css/style.css   js/validate.js
        `-- WEB-INF/
            |-- web.xml
            |-- lib/mysql-connector-j-8.4.0.jar
            `-- views/  home, reportItem, search, itemDetails,
                        myReports, myClaims, admin/(dashboard,
                        items, claims)  .jsp
```

6. Database Design

6.1 Database

A MySQL database named `**campus_lost_found**` stores all data. It has three tables. The column names and data types are exactly as given in the suggested design; primary keys use AUTO_INCREMENT, and foreign keys keep the data consistent.

6.2 Table 1: users

Field	Data Type	Description
user_id	INT	Primary key
full_name	VARCHAR(100)	User name
email	VARCHAR(100)	Email address
password	VARCHAR(255)	Password hash
mobile	VARCHAR(15)	Contact number
user_role	VARCHAR(20)	User or administrator

6.3 Table 2: items

Field	Data Type	Description
item_id	INT	Primary key
user_id	INT	Foreign key
item_name	VARCHAR(100)	Item name
category	VARCHAR(50)	Item category
description	TEXT	Item description
report_type	VARCHAR(10)	Lost or Found
location	VARCHAR(150)	Lost or found location
report_date	DATE	Date of incident
status	VARCHAR(30)	Current item status

6.4 Table 3: claims

Field	Data Type	Description
claim_id	INT	Primary key
item_id	INT	Foreign key
user_id	INT	Claimant ID
ownership_details	TEXT	Ownership explanation

Field	Data Type	Description
claim_date	DATE	Date of claim
claim_status	VARCHAR(30)	Pending, Approved or Rejected
admin_remarks	TEXT	Verification remarks

6.5 Entity-Relationship Diagram

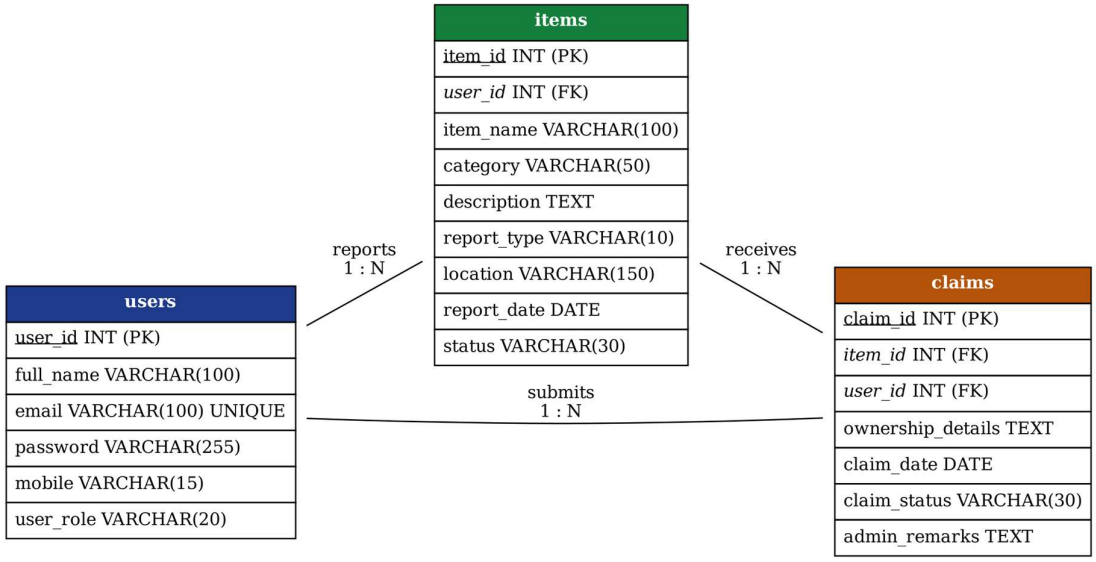


Figure 6.1: ER diagram of the campus_lost_found database

Relationship	Type	Foreign key
A user reports items	One-to-Many	items.user_id → users.user_id
An item receives claims	One-to-Many	claims.item_id → items.item_id
A user submits claims	One-to-Many	claims.user_id → users.user_id

6.6 Database Creation SQL Script

```
CREATE DATABASE campus_lost_found;
USE campus_lost_found;

CREATE TABLE users (
    user_id    INT AUTO_INCREMENT PRIMARY KEY,
    full_name  VARCHAR(100) NOT NULL,
    email      VARCHAR(100) NOT NULL UNIQUE,
    password   VARCHAR(255) NOT NULL,
    mobile     VARCHAR(15),
    user_role  VARCHAR(20)  NOT NULL DEFAULT 'USER'
);

CREATE TABLE items (
    item_id      INT AUTO_INCREMENT PRIMARY KEY,
    user_id      INT          NOT NULL,
    item_name    VARCHAR(100) NOT NULL,
    category     VARCHAR(50)  NOT NULL,
    description  TEXT,
```

```
    report_type  VARCHAR(10)  NOT NULL,
    location     VARCHAR(150) NOT NULL,
    report_date  DATE         NOT NULL,
    status       VARCHAR(30)  NOT NULL DEFAULT 'Open',
    FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE CASCADE
);

CREATE TABLE claims (
    claim_id           INT AUTO_INCREMENT PRIMARY KEY,
    item_id            INT  NOT NULL,
    user_id            INT  NOT NULL,
    ownership details  TEXT NOT NULL,
    claim_date         DATE NOT NULL,
    claim_status       VARCHAR(30) NOT NULL DEFAULT 'Pending',
    admin_remarks      TEXT,
    FOREIGN KEY (item_id) REFERENCES items(item_id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- Stored procedure for the admin summary (CallableStatement)
DELIMITER //
CREATE PROCEDURE sp_item_summary(OUT total INT, OUT lost INT,
        OUT found INT, OUT closed INT, OUT pending INT)
BEGIN
    SELECT COUNT(*) INTO total   FROM items;
    SELECT COUNT(*) INTO lost    FROM items WHERE report_type='Lost';
    SELECT COUNT(*) INTO found   FROM items WHERE report_type='Found';
    SELECT COUNT(*) INTO closed  FROM items WHERE status='Closed';
    SELECT COUNT(*) INTO pending FROM claims WHERE claim_status='Pending';
END //
DELIMITER ;

-- Administrator account (password admin123, stored as SHA-256)
INSERT INTO users (full_name, email, password, mobile, user_role)
VALUES ('Portal Admin', 'admin@campus.edu',
 '240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9',
 '9000000001', 'ADMIN');
```

6.7 Status Values

Column	Values	Meaning
items.report_type	Lost, Found	Type of report
items.status	Open, Claim Pending, Closed	Closed means recovered and returned
claims.claim_status	Pending, Approved, Rejected	Administrator decision
users.user_role	USER, ADMIN	Access level

7. Implementation

This chapter explains how each implementation requirement (A to F) is met, with short parts of the actual source code.

7.1 HTML Forms and HTTP

The front end uses plain HTML forms. `register.html` is a static page, while the other forms are inside JSP pages. Each form uses the correct HTTP method: **POST** for anything that changes data (register, login, report, claim, admin decisions) and **GET** for reading data (search, item details, lists), so that a search result can be bookmarked.

```
<!-- register.html : data goes to RegisterServlet using POST -->
<form action="register" method="post"
      onsubmit="return validateRegister(this)">
    <input type="text"     name="fullName" required>
    <input type="email"    name="email"    required>
    <input type="tel"      name="mobile"   maxlength="10" required>
    <input type="password" name="password" required>
    <input type="password" name="confirm"  required>
    <button type="submit">Register</button>
</form>

<!-- search.jsp : GET form, values appear in the URL -->
<form action="search" method="get">
    <input type="text" name="keyword">
    <select name="category"> ... </select>
    <select name="type"> ... </select>
</form>
```

Client-side validation is written in `js/validate.js`. It stops the form before it reaches the server when the data is wrong. Version 2 shows each message inside the form, next to the field, through a small helper fail(form, field, message); the servlets check the same rules again on the server (Section 7.9).

```
function validateRegister(f) {
    if (f.fullName.value.trim().length < 3) {
        return fail(f, "fullName", "Enter your full name (at least 3 letters)."); }
    if (!/^[0-9]{10}$/.test(f.mobile.value)) {
        return fail(f, "mobile", "Mobile number must be exactly 10 digits."); }
    if (f.password.value.length < 6) {
        return fail(f, "password", "Use a password of at least 6 characters."); }
    if (f.password.value !== f.confirm.value) {
        return fail(f, "confirm", "The two passwords don't match."); }
    return true;
}
```

HTTP feature	Where it is used
GET	/home, /search, /item?id=n, /myReports, /claims, /admin?view=…
POST	/register, /login, /report, /claims, /admin
302 Redirect	After every successful POST (Post-Redirect-Get pattern)
403 Forbidden	A normal user opening /admin

HTTP feature	Where it is used
404 Not Found	Unknown page or item id

7.2 Java Servlets

Every servlet extends `HttpServlet` and overrides `doGet()` and/or `doPost()`. Most are mapped with the `@WebServlet` annotation. `LoginServlet` is mapped in `web.xml` to show the deployment descriptor, a `ServletConfig` init parameter and `load-on-startup`.

```
<servlet>
    <servlet-name>LoginServlet</servlet-name>
    <servlet-class>com.campus.lostfound.controller.LoginServlet</servlet-class>
    <init-param>
        <param-name>sessionTimeoutMinutes</param-name>
        <param-value>30</param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>LoginServlet</servlet-name>
    <url-pattern>/login</url-pattern>
</servlet-mapping>
```

Servlet Life Cycle

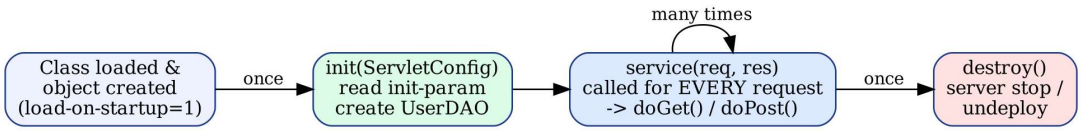


Figure 7.1: Servlet life cycle shown by LoginServlet

```
public void init(ServletConfig config) throws ServletException {
    super.init(config);
    userDAO = new UserDAO();
    sessionTimeout = Integer.parseInt(
            config.getInitParameter("sessionTimeoutMinutes"));
    System.out.println("[LoginServlet] init() called - servlet loaded");
}
protected void service(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
    System.out.println("[LoginServlet] service() -> " + req.getMethod());
    super.service(req, res);          // calls doGet() or doPost()
}
public void destroy() {
    System.out.println("[LoginServlet] destroy() called - servlet unloaded");
}
```

The Tomcat log shows the life cycle while the application runs: `init()` once at start-up, `service()` for every request, and `destroy()` when Tomcat stops.

Processing a Lost / Found Report

```
@WebServlet("/report")
public class ReportItemServlet extends HttpServlet {
    private final ItemDAO itemDAO = new ItemDAO();


    protected void doPost(HttpServletRequest request, HttpServletResponse response)
```

```
            throws ServletException, IOException {
        User user = SessionUtil.currentUser(request);
        if (user == null) { response.sendRedirect("login"); return; }

        Item it = new Item();
        it.setUserId(user.getUserId());
        it.setItemName(request.getParameter("itemName").trim());
        it.setCategory(request.getParameter("category"));
        it.setDescription(request.getParameter("description").trim());
        String type = request.getParameter("reportType");
        it.setReportType("Found".equals(type) ? "Found" : "Lost");
        it.setLocation(request.getParameter("location").trim());
        it.setReportDate(request.getParameter("reportDate"));

        itemDAO.addItem(it);
        response.sendRedirect("myReports?msg=saved");
    }
}
```

Administrator Operations

`AdminServlet` handles all administrator work in one controller. It first checks the role and returns **403 Forbidden** for anyone who is not an administrator. The `view` parameter selects the page (dashboard, items, claims) and the `action` parameter selects the operation (status, delete, decide).

```
if (!isAdmin(request)) {
    response.sendError(HttpServletResponse.SC_FORBIDDEN, "Administrators only");
    return;
}
String action = request.getParameter("action");
if ("status".equals(action)) {
    itemDAO.updateStatus(toInt(request.getParameter("itemId")),
                         request.getParameter("status"));
} else if ("delete".equals(action)) {
    itemDAO.deleteItem(toInt(request.getParameter("itemId")));
} else if ("decide".equals(action)) {
    claimDAO.decideClaim(toInt(request.getParameter("claimId")), status,
                         request.getParameter("remarks"));
}
```

7.3 JSP

JSP pages build the HTML using the beans that the servlet places in the request. The project uses every JSP element from the syllabus.

JSP feature	Example in the project
Page directive	`<%@ page import="java.util.List, com.campus.lostfound.bean.*" errorPage="/error.jsp" %>`
Include directive	`<%@ include file="/includes/footer.jsp" %>`
Expression	`<%= it.getItemName() %>`
Scriptlet	`<% for (Item it : items) { %> … <% } %>`
Declaration	`<%! private int pageVisits = 0; %>` and helper method `badge()`
jspInit / jspDestroy	Declared in `home.jsp`
jsp:include	`<jsp:include page="/includes/header.jsp" />` on every page

JSP feature	Example in the project
jsp:forward	`index.jsp` → `<jsp:forward page="/home" />`
Implicit objects	request, response, session, application, out, config, page, exception

```
<%!
    // declarations: become fields and methods of the generated servlet
    private int pageVisits = 0;
    public void jspInit()    { System.out.println("[home.jsp] jspInit() called"); }
    public void jspDestroy() { System.out.println("[home.jsp] jspDestroy() called"); }
%>
<jsp:include page="/includes/header.jsp" />
<%
    pageVisits++;
    User user = (User) session.getAttribute("user");
    List<Item> items = (List<Item>) request.getAttribute("recentItems");
%>
<h2>Welcome, <%= HtmlUtil.esc(user.getFullName()) %></h2>
<% for (Item it : items) { %>
    <a class="card" href="item?id=<%= it.getItemId() %>">
        <h4><%= HtmlUtil.esc(it.getItemName()) %></h4>
        <p><%= it.getCategory() %> &middot; <%= it.getLocation() %></p>
    </a>
<% } %>
<%@ include file="/includes/footer.jsp" %>
```

Implicit objects used: `request` (attributes, parameters, cookies), `response` (`encodeURL()` for URL rewriting), `session` (logged-in user), `application` (`getInitParameter("appName")` from `web.xml`), `config` (`getServletName()`), `out` (used by every expression), `page` (the current JSP instance) and `exception` (in `error.jsp`, which has `isErrorPage="true"`).

7.4 JavaBeans and MVC Integration

Three JavaBeans hold the data of the three tables. Each bean has private fields, public getters and setters, a public no-argument constructor and implements `Serializable` (so it can be stored in the session).

```
public class Item implements Serializable {
    private int itemId;
    private int userId;
    private String itemName;
    private String category;
    private String description;
    private String reportType;   // Lost or Found
    private String location;
    private String reportDate;
    private String status;       // Open, Claim Pending, Closed

    public Item() { }
    public String getItemName() { return itemName; }
    public void setItemName(String itemName) { this.itemName = itemName; }
    // ... other getters and setters
}
```

How MVC is integrated — `SearchServlet` is a good example of the three layers working together:

```
@WebServlet("/search")
public class SearchServlet extends HttpServlet {
    private final ItemDAO itemDAO = new ItemDAO();              // Model

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {                // Controller
        List<Item> results = itemDAO.search(request.getParameter("keyword"),
                request.getParameter("category"), request.getParameter("type"));
        request.setAttribute("results", results);
        request.getRequestDispatcher("/WEB-INF/views/search.jsp")
               .forward(request, response);                       // View
    }
}
```

7.5 JDBC

`DBConnection` loads the MySQL driver once and returns a new `Connection` when asked. Every DAO method uses **try-with-resources**, so the Connection, Statement and ResultSet are closed automatically, even when an exception occurs.

```
public class DBConnection {
    private static final String URL = "jdbc:mysql://localhost:3306/campus_lost_found";
    private static final String USER = "root";
    private static final String PASSWORD = "root";

    static {
        try { Class.forName("com.mysql.cj.jdbc.Driver"); }      // register driver
        catch (ClassNotFoundException e) { System.out.println("Driver not found"); }
    }
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

JDBC interface / operation	Method in the project
Connection	`DBConnection.getConnection()`
Statement	`ItemDAO.allItems()`, `recentItems()` (fixed queries)
PreparedStatement	Register, login, add item, search, update status, delete, claims
CallableStatement	`ItemDAO.summary()` calls `sp_item_summary`
ResultSet	Converting rows into Item, User and Claim beans
INSERT	`UserDAO.register()`, `ItemDAO.addItem()`, `ClaimDAO.addClaim()`
SELECT	`login()`, `search()`, `getById()`, `itemsByUser()`, `allClaims()`
UPDATE	`ItemDAO.updateStatus()`, `ClaimDAO.decideClaim()`
DELETE	`ItemDAO.deleteItem()`

PreparedStatement — login:

```
String sql = "SELECT * FROM users WHERE email = ? AND password = ?";
try (Connection con = DBConnection.getConnection();
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, email);
    ps.setString(2, PasswordUtil.hash(password));
```

```
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) { /* build and return the User bean */ }
    }
}
```

CallableStatement — administrator summary:

```
try (Connection con = DBConnection.getConnection();
     CallableStatement cs = con.prepareCall("{call sp_item_summary(?, ?, ?, ?, ?)}"))
{
    for (int i = 1; i <= 5; i++) cs.registerOutParameter(i, Types.INTEGER);
    cs.execute();
    return new int[] { cs.getInt(1), cs.getInt(2), cs.getInt(3),
                       cs.getInt(4), cs.getInt(5) };
}
```

Transaction management — approving a claim. Approving a claim changes three things: the claim becomes Approved, the item becomes Closed, and other pending claims on the item become Rejected. All three must succeed together, so auto-commit is switched off and the changes are committed at the end. If any statement fails, `rollback()` cancels everything.

```
Connection con = null;
try {
    con = DBConnection.getConnection();
    con.setAutoCommit(false);                        // start transaction
    // 1. UPDATE claims SET claim_status=?, admin_remarks=? WHERE claim_id=?
    // 2. UPDATE items  SET status='Closed' WHERE item_id=?
    // 3. UPDATE claims SET claim_status='Rejected'
    //       WHERE item_id=? AND claim_status='Pending'
    con.commit();                                    // save all together
} catch (SQLException e) {
    if (con != null) con.rollback();                 // undo all
    throw new RuntimeException("Claim update failed", e);
} finally {
    if (con != null) con.close();
}
```

Exception handling: SQL errors are wrapped in a RuntimeException and passed to the container. The `<error-page>` entries in `web.xml` send them to `error.jsp`, which shows a friendly message and logs the real error using `application.log()`.

7.6 Session Management

Requirement	Implementation
Authenticated session	After login, the `User` bean is stored with `session.setAttribute("user", user)`; timeout 30 minutes from the ServletConfig parameter
Session check	Every protected servlet calls `SessionUtil.currentUser(request)`, which uses `getSession(false)` and redirects to login when it is null
Admin restriction	`AdminServlet` checks `user.isAdmin()` and returns 403 otherwise
Logout	`LogoutServlet` calls `session.invalidate()` and redirects to login
Session fixation	The old session is invalidated and a new one is created at login
Cookie	`lastEmail` cookie (7 days, HttpOnly) pre-fills the login email

Requirement	Implementation
URL rewriting	Navigation links use `response.encodeURL()`, so the session works even when cookies are blocked (the URL gets `;jsessionid=…`)

```
HttpSession old = request.getSession(false);
if (old != null) old.invalidate();                // new session after login
HttpSession session = request.getSession(true);
session.setAttribute("user", user);
session.setMaxInactiveInterval(sessionTimeout * 60);

Cookie cookie = new Cookie("lastEmail", email);
cookie.setMaxAge(7 * 24 * 60 * 60);
cookie.setHttpOnly(true);
response.addCookie(cookie);
response.sendRedirect(response.encodeRedirectURL(user.isAdmin() ? "admin" : "home"));
```

7.7 Security Measures

- **Password hashing:** passwords are converted to SHA-256 using `MessageDigest` before they are saved or compared.
- **SQL injection:** every value typed by a user is passed through a `PreparedStatement` parameter.
- **Script injection (XSS):** all user text printed in JSP goes through `HtmlUtil.esc()`.
- **Protected views:** JSP views live in `WEB-INF/views` and cannot be opened directly.
- **Business rules:** users cannot claim their own report, a closed item, or the same item twice; new accounts always get the USER role.

7.8 User Interface Design (Version 2)

The second version of the portal gives every page a consistent, professional design while keeping the same JSP pages and MVC flow. All styling lives in one stylesheet, `css/style.css`, built on design tokens: named CSS custom properties for colours, type, spacing and corner radius that every component reuses. No UI framework is used, so the front end stays within HTML, CSS and JavaScript as the assignment requires.

- **Branding:** The GM University logo appears in the top bar, on the login, register and error pages, in the footer and as the browser-tab icon.
- **Colour with meaning:** one blue for actions and links; red always means Lost and green always means Found; grey, amber and green chips show Open, Claim Pending and Closed.

- **Tag cards:** every item is shown as a lost-and-found tag with a reference number such as LF-0009, built from item_id. The same card is used on the home page and in search results.
- **Inline validation:** errors are shown inside the form next to the field instead of alert() pop-ups. validate.js keeps the same function names, so the forms did not change.
- **Responsive and dark mode:** the layout adapts to phones (collapsible menu, stacked tables) and follows the system's dark mode (Figures 8.17 and 8.18).
- **Footer:** a footer with portal links, project details and the developer's GitHub signature appears on every page.

7.9 Server-Side Validation and Business Rules

Client-side checks can be skipped by turning JavaScript off or by sending a request directly, so version 2 repeats every important rule in the servlets. Each check redirects back with a clear message, or returns an HTTP error page.

- **ClaimServlet:** a user can claim an item only once (ClaimDAO.hasClaimed() plus a UNIQUE (item_id, user_id) key on the claims table); administrators cannot file claims; the proof must be at least 15 characters.
- **AdminServlet:** the item status must be one of Open, Claim Pending or Closed, otherwise the servlet returns HTTP 400 Bad Request.
- **ReportItemServlet:** the name, category, location and date are required, the category must be from the list, and the date cannot be in the future.

```
boolean notAllowed = item == null
        || user.isAdmin()                          // admins only review
        || item.getUserId() == user.getUserId()    // own report
        || "Closed".equals(item.getStatus())       // already returned
        || claimDAO.hasClaimed(itemId, userId);    // one claim per item
if (notAllowed) {
    response.sendRedirect("item?id=" + itemId + "&error=notallowed");
    return;
}
if (details.length() < 15) {                       // CM-02: enough proof
    response.sendRedirect("item?id=" + itemId + "&error=short");
    return;
}
```

8. Application Screens

The following screens were captured from the running application on Apache Tomcat 10.1 with the MySQL database.

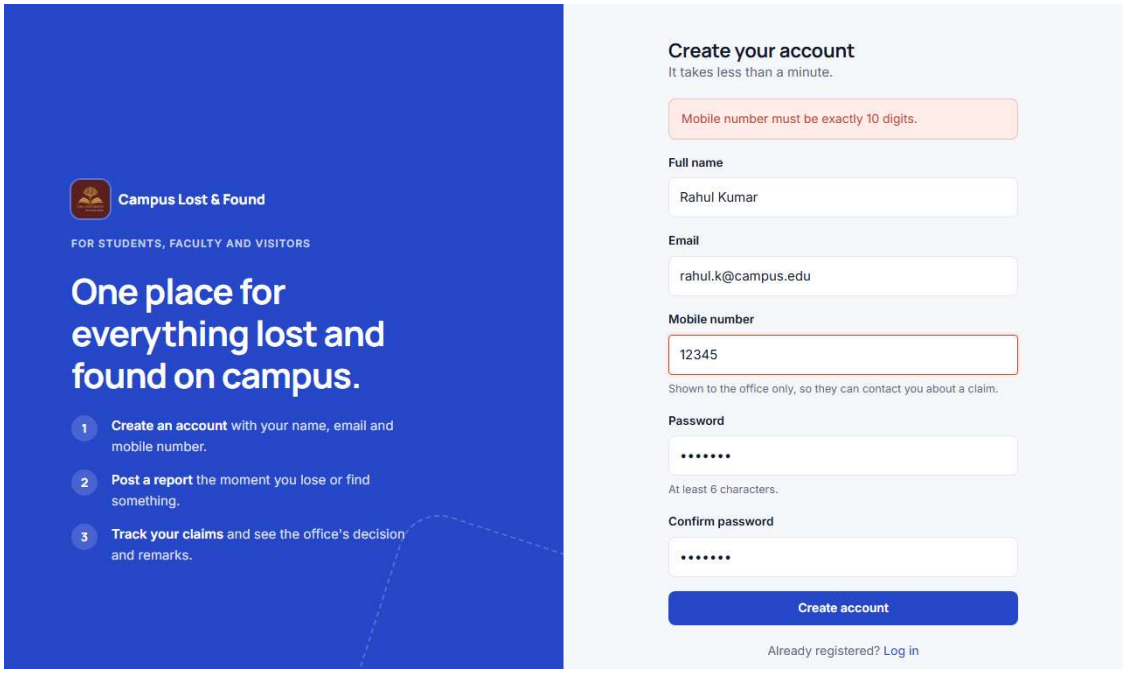


Figure 8.1: User registration form with client-side validation

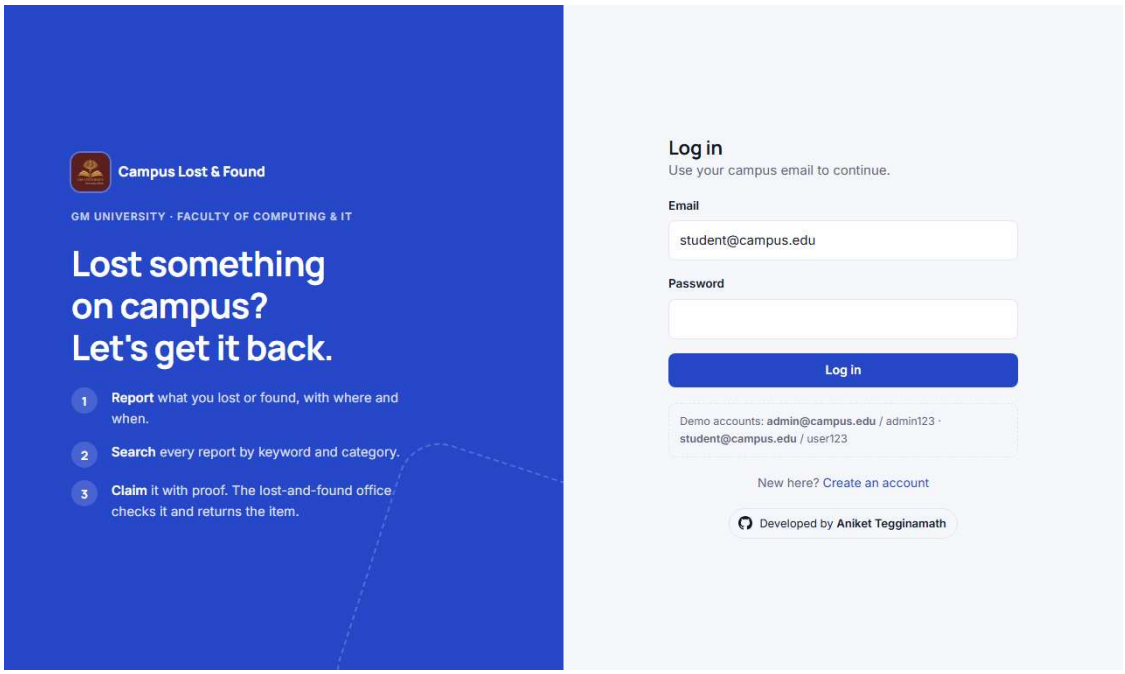


Figure 8.2: Login page (email pre-filled from cookie)

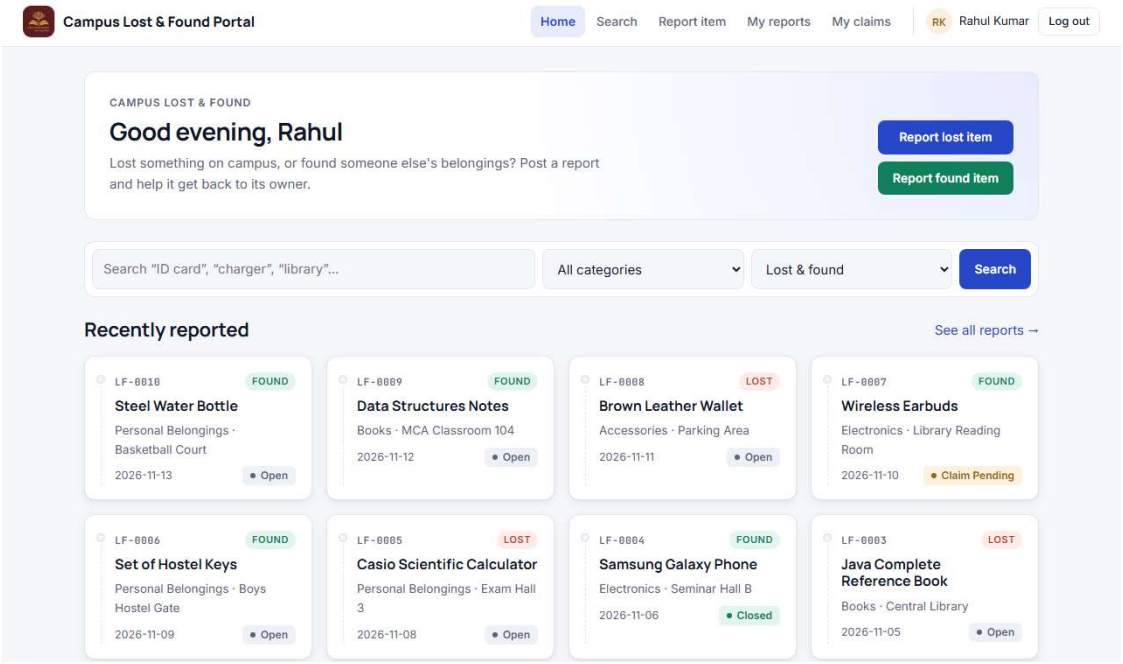


Figure 8.3: Home page showing recently reported items

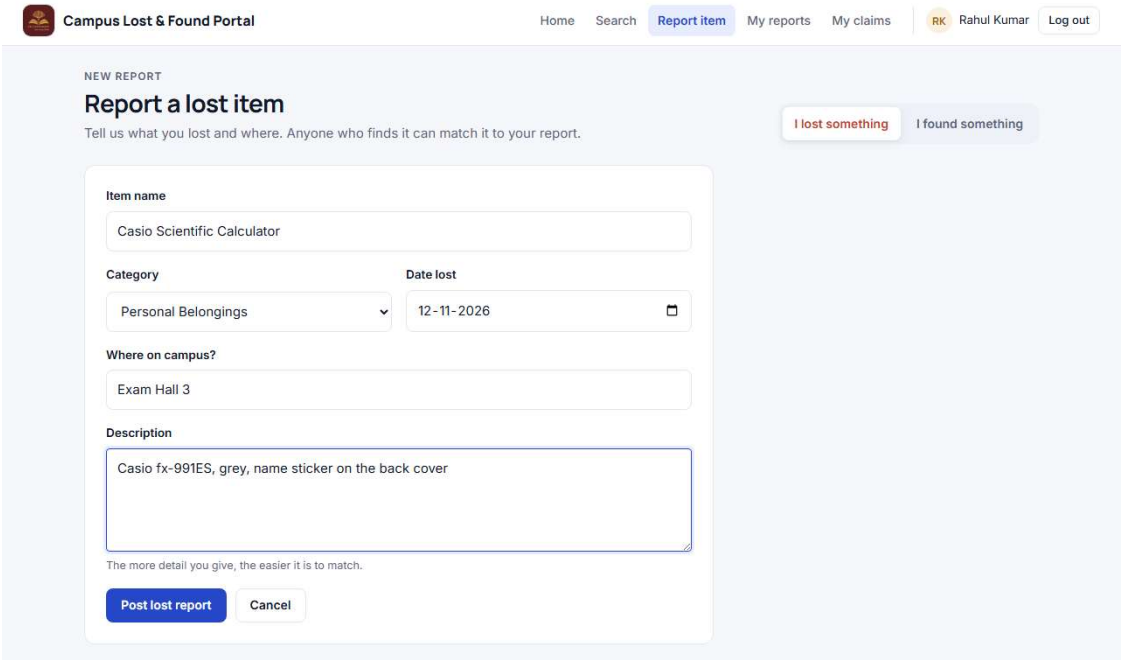


Figure 8.4: Report Lost Item form

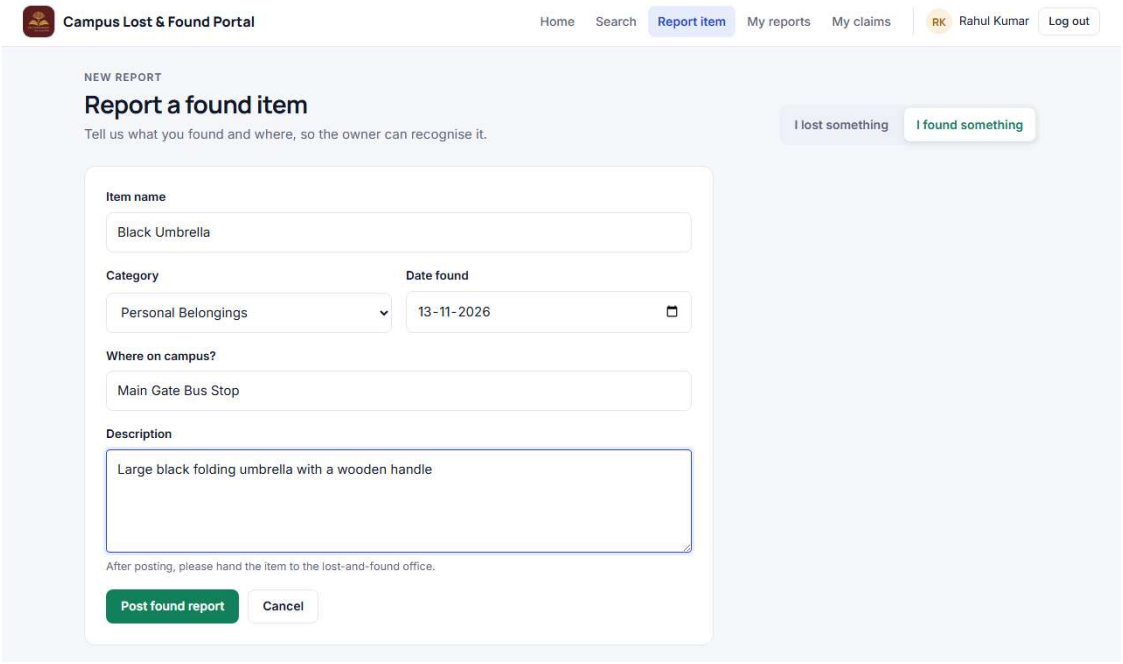


Figure 8.5: Report Found Item form

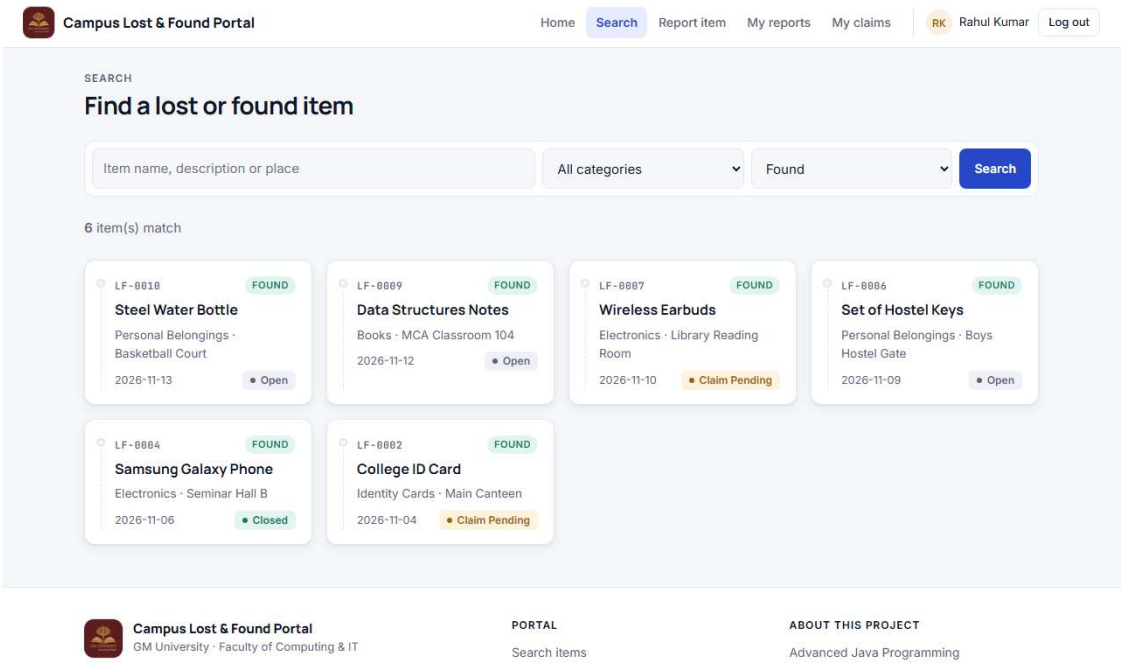


Figure 8.6: Search by keyword, category and type

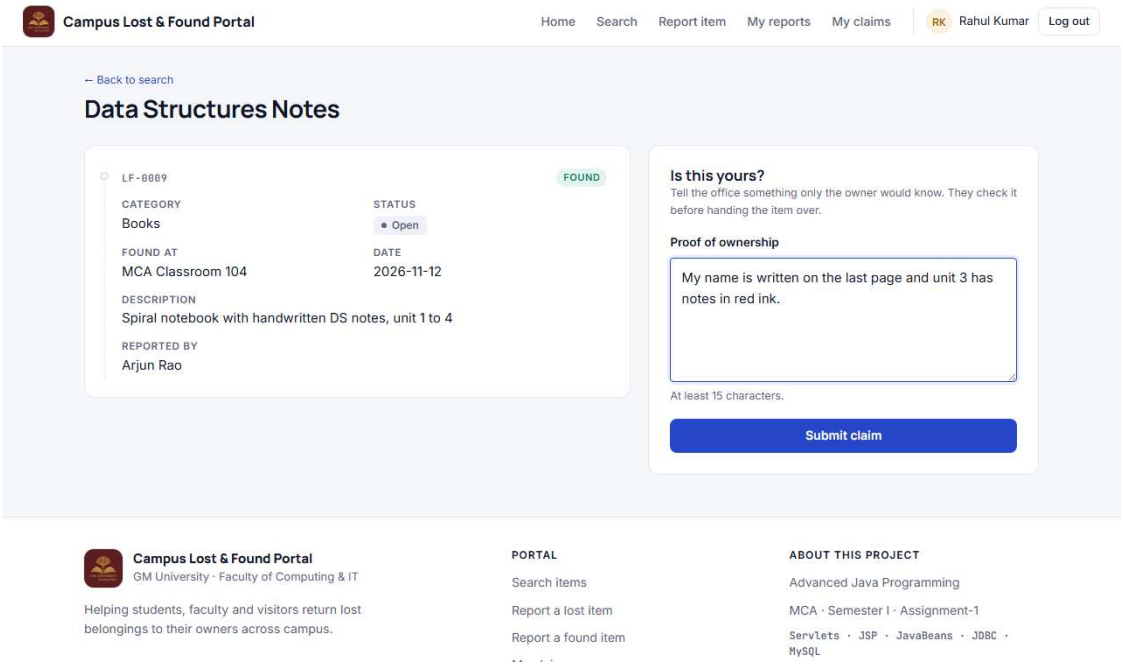


Figure 8.7: Item details with ownership claim form

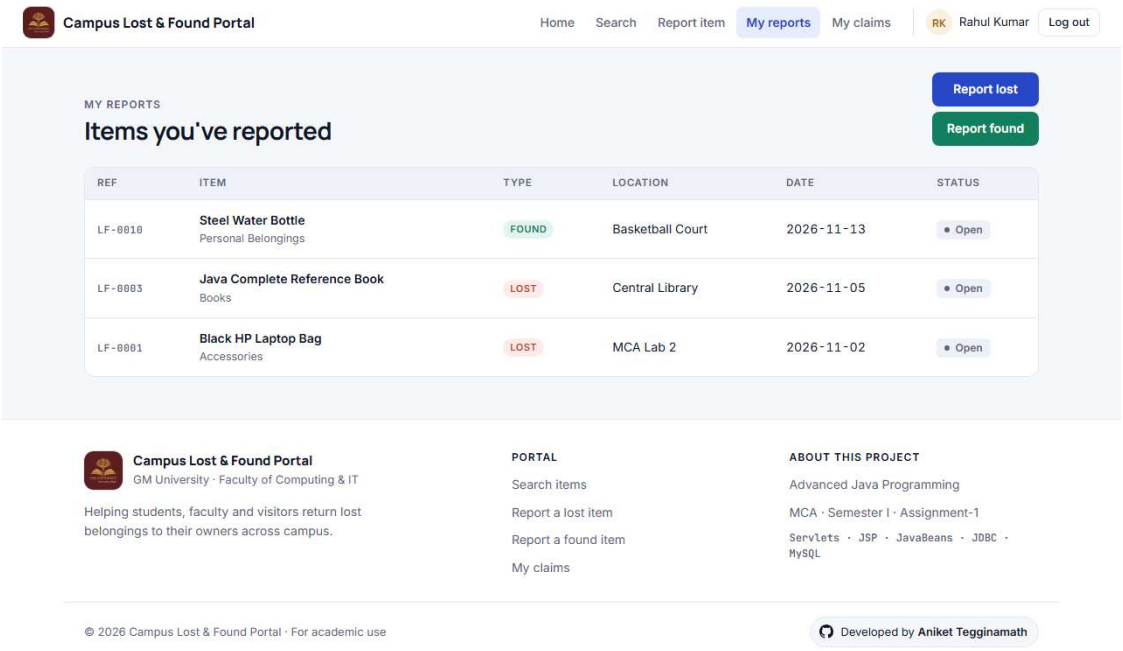


Figure 8.8: My Reports page

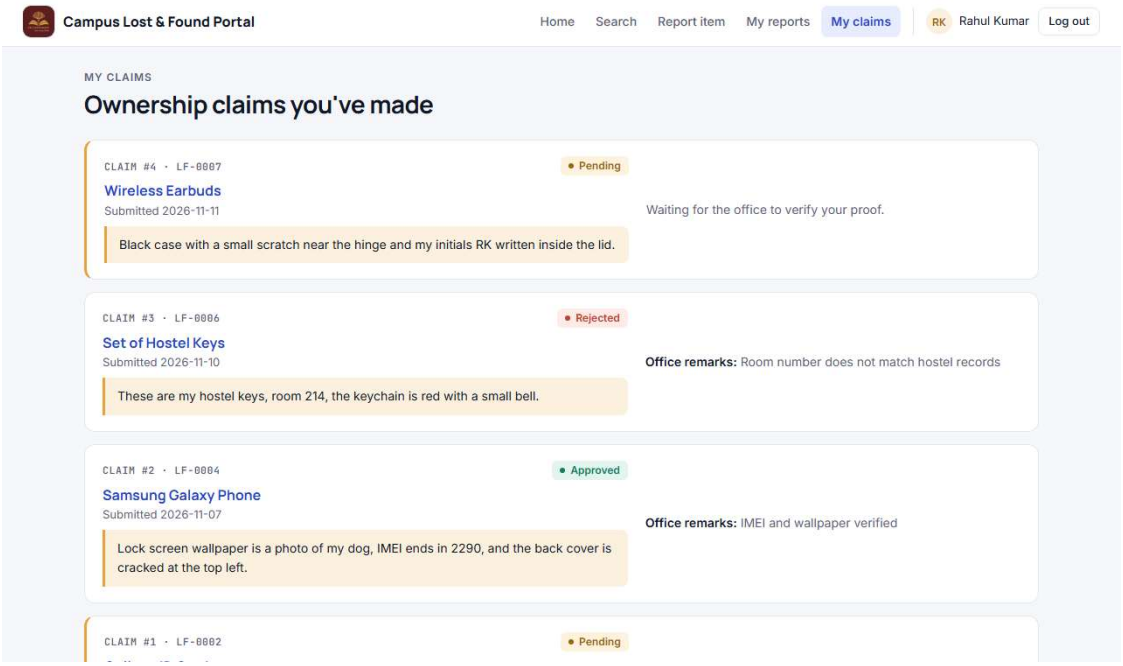


Figure 8.9: My Claims page showing Approved and Rejected claims with remarks

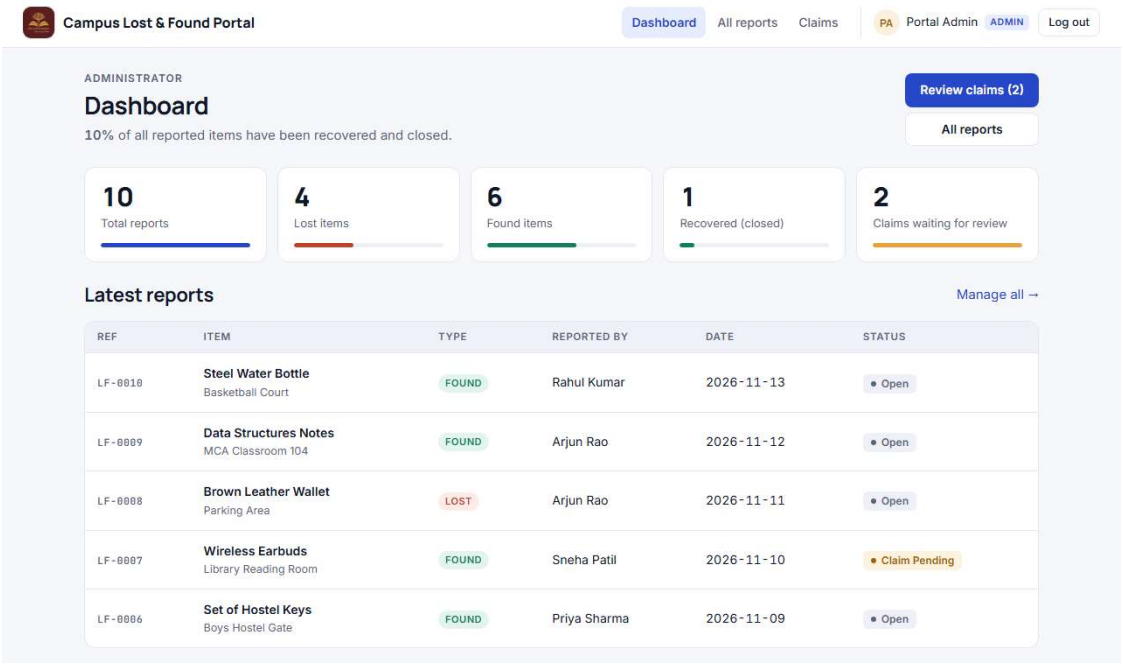


Figure 8.10: Administrator dashboard with summary (CallableStatement)

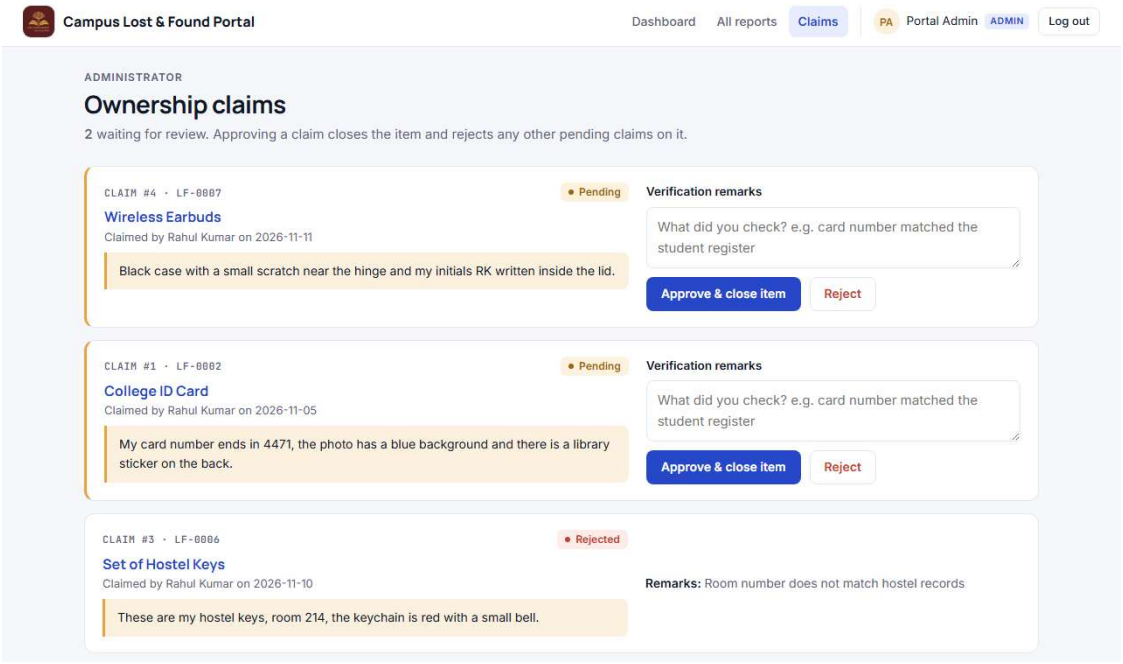


Figure 8.11: Administrator — pending ownership claims

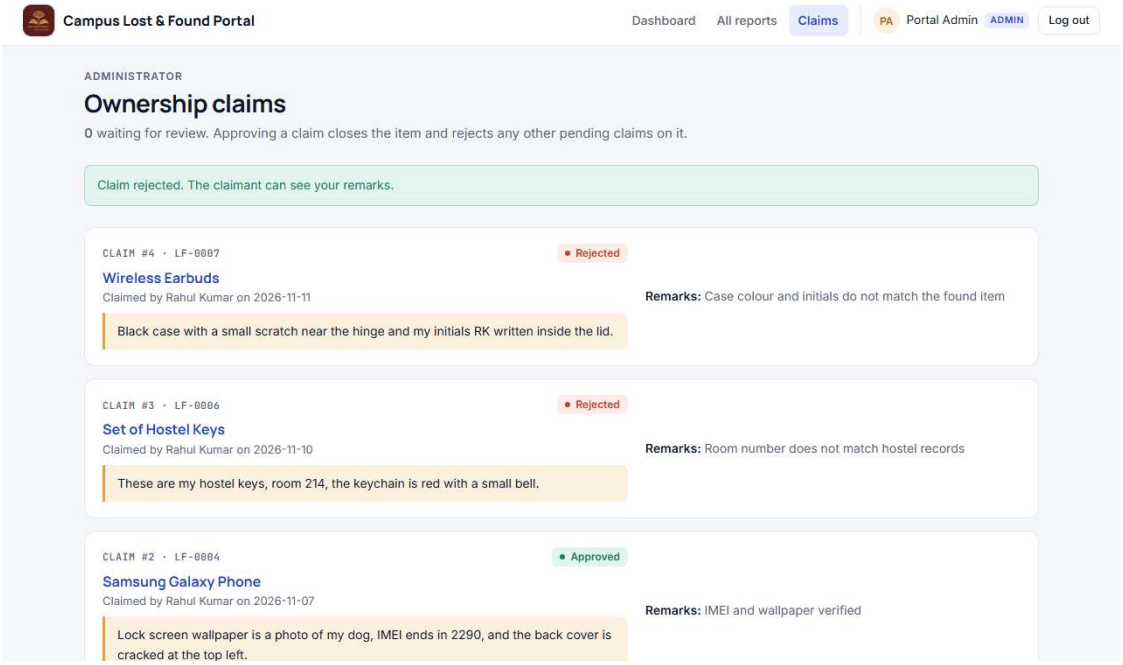


Figure 8.12: Administrator — claims after approval and rejection

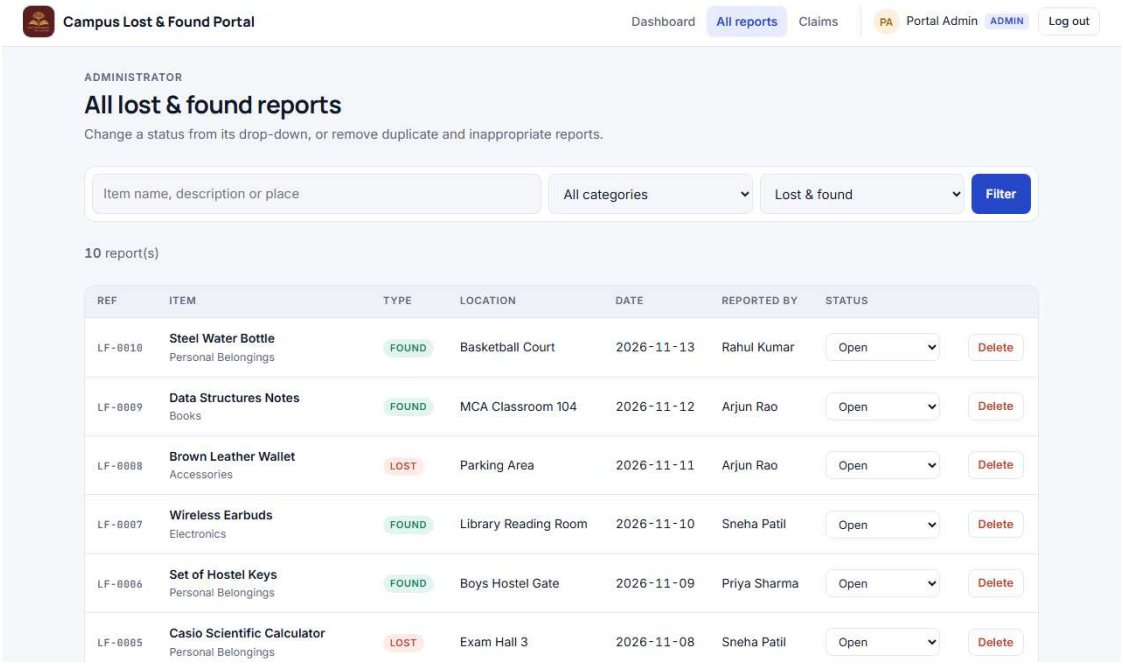


Figure 8.13: Administrator — all reports with filter, status update and delete

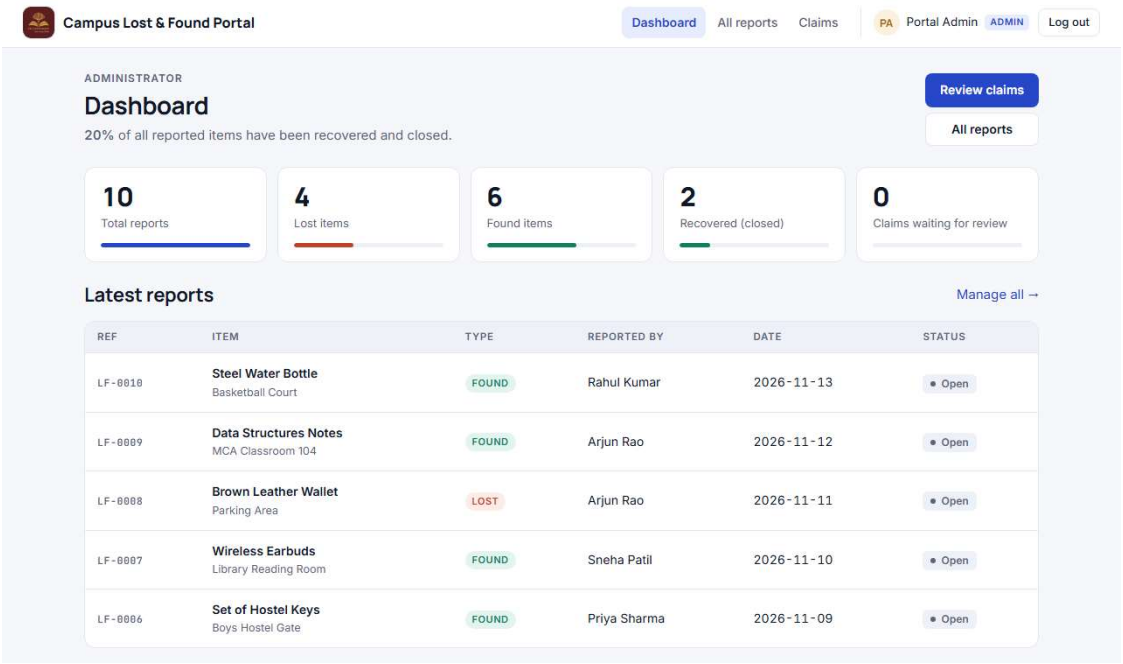


Figure 8.14: Dashboard after recovery — item marked Closed

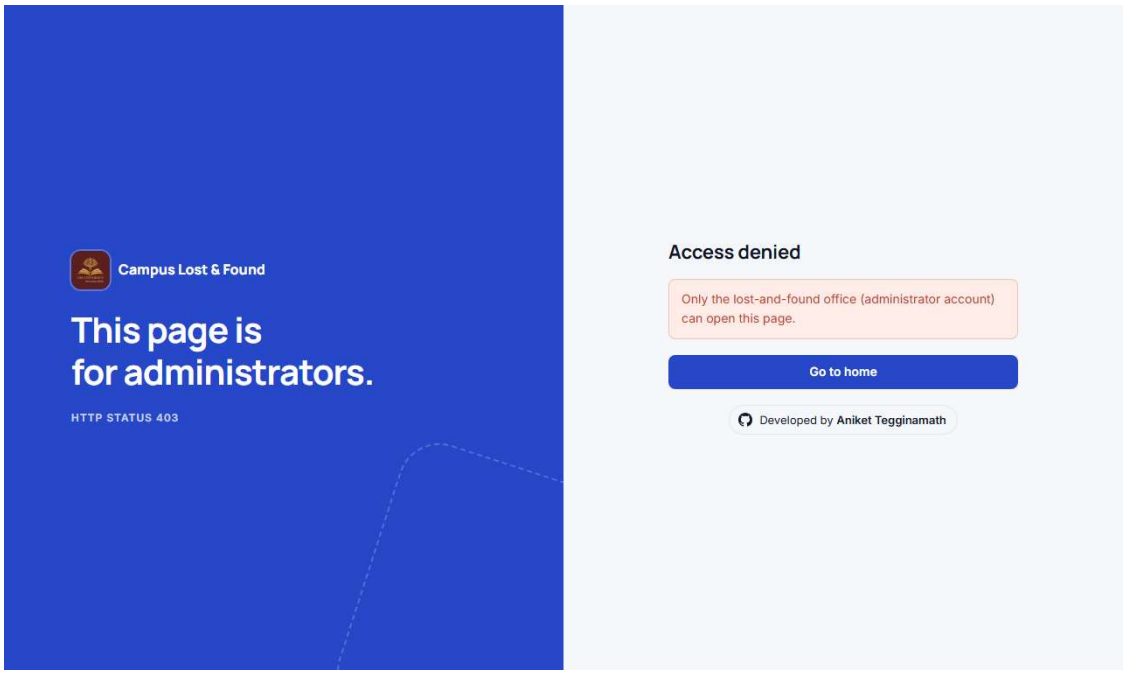


Figure 8.15: Access denied (HTTP 403) when a user opens an admin page

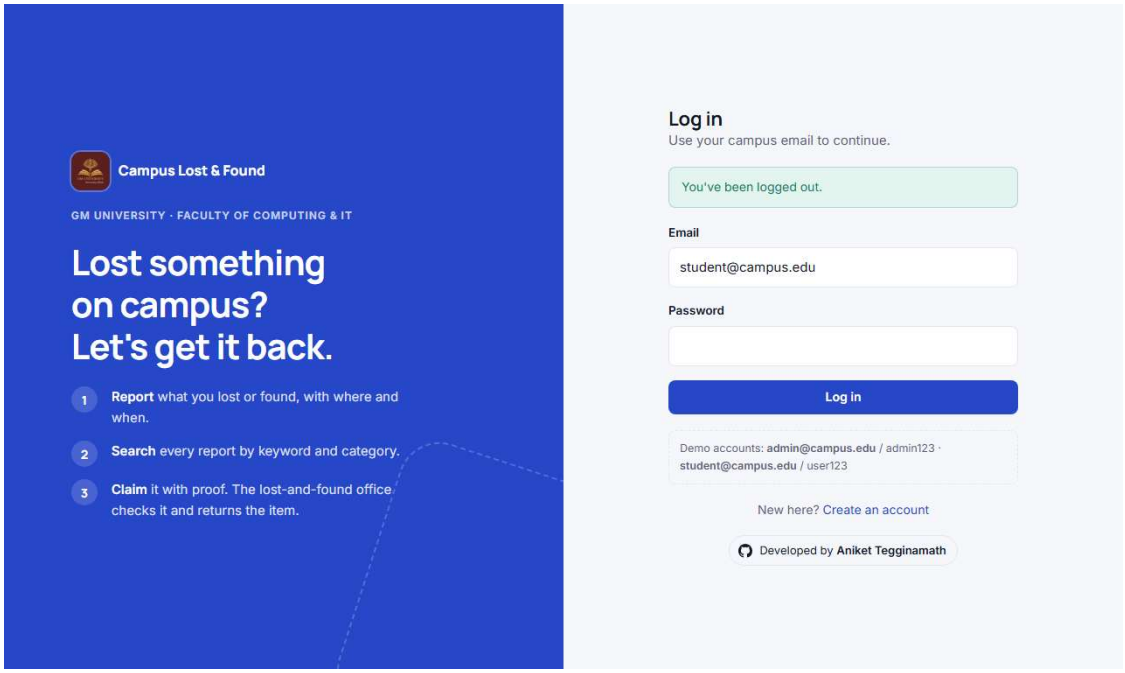


Figure 8.16: Logout with session invalidation

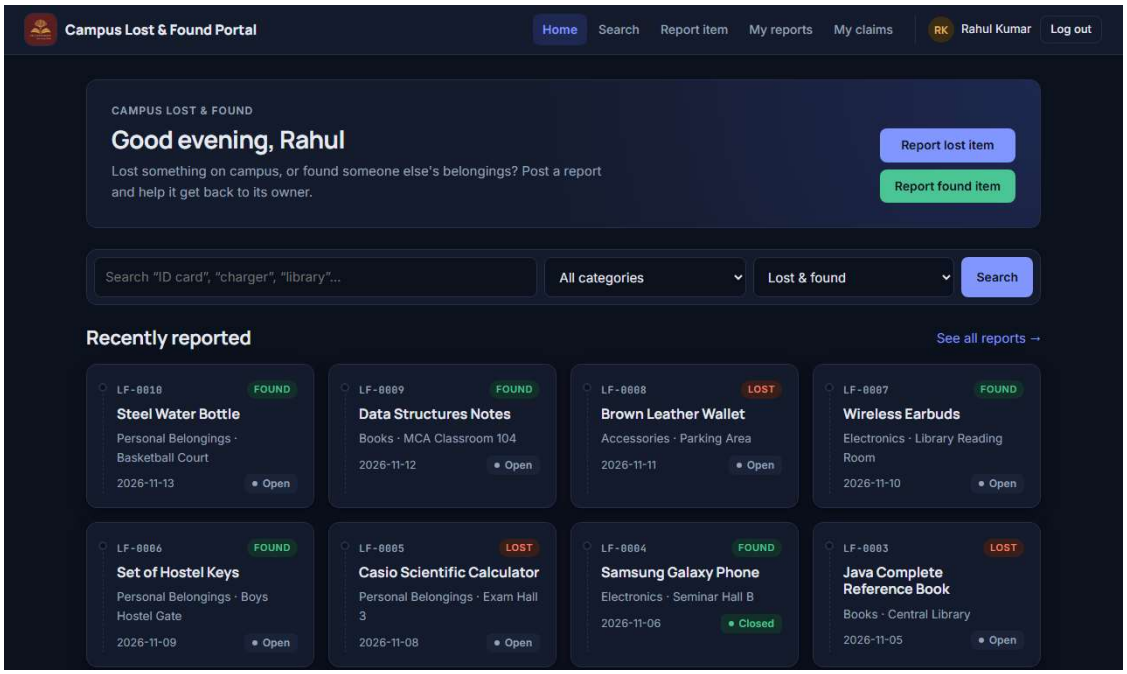


Figure 8.17: Home page in dark mode (follows the system theme)

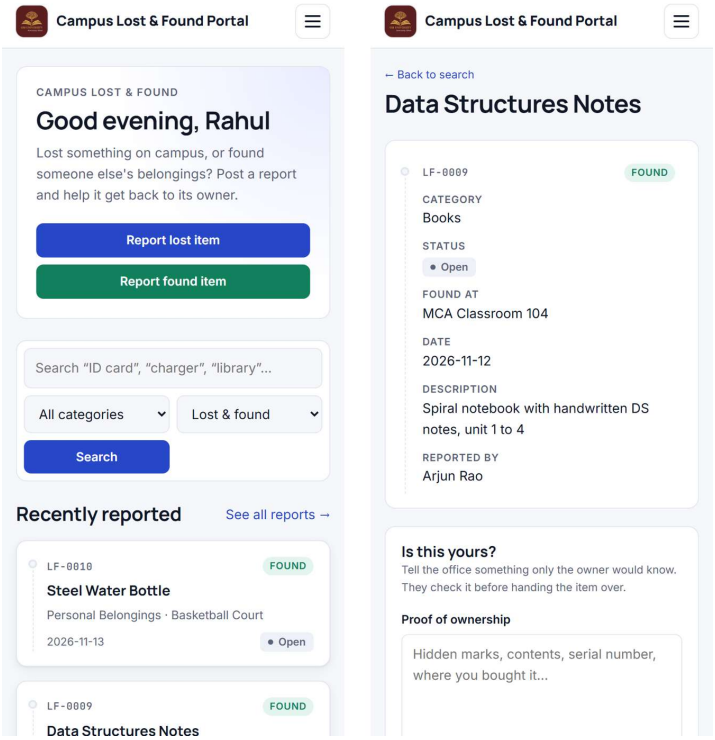


Figure 8.18: Phone layout: home page and item details with the claim form

9. Testing

9.1 Testing Approach

The application was tested module by module (unit level), then as a whole (integration level) by following the full path of an item from report to recovery. Both valid and invalid inputs were tried. The database was checked after each operation using MySQL queries.

9.2 Test Cases and Results

TC	Module	Test case	Input	Expected result	Result
1	User	Register new user	Valid name, email, 10-digit mobile, password	Account created, redirect to login with success message	Pass
2	User	Register with existing email	student@campus.edu	"Email already registered" message	Pass
3	User	Client validation	Mobile "12345"	Inline error "Mobile number must be exactly 10 digits."; form not sent	Pass
4	User	Login with valid details	student@campus.edu / user123	Home page with welcome message	Pass
5	User	Login with wrong password	wrong password	"Invalid email or password"	Pass
6	Item	Report lost item	Calculator, Personal Belongings, Exam Hall 3	Row in items, status Open	Pass
7	Item	Report found item	Phone, Electronics, Seminar Hall	Row in items, type Found	Pass
8	Item	Future date	Date after today	Inline error "The date can't be in the future."	Pass
9	Search	Keyword search	"bag"	Only the laptop bag listed	Pass
10	Search	Category filter	Electronics	Only electronics items	Pass
11	Item	View details	/item?id=4	All fields and reporter name shown	Pass

TC	Module	Test case	Input	Expected result	Result
12	Item	Invalid item id	/item?id=999	404 error page	Pass
13	Claim	Submit claim	Proof text (more than 15 characters)	Claim Pending; item becomes Claim Pending	Pass
14	Claim	Claim own item	Reporter opens own item	Claim form hidden; direct POST refused	Pass
15	Claim	View claim status	My Claims page	Status and admin remarks shown	Pass
16	Admin	Admin login	admin@campus.edu / admin123	Admin dashboard with counts	Pass
17	Admin	Approve claim	Remarks "IMEI verified"	Claim Approved, item Closed, others Rejected	Pass
18	Admin	Reject claim	Remarks given	Claim Rejected; item Open again	Pass
19	Admin	Update status	Change to Closed	Status updated in table	Pass
20	Admin	Delete duplicate	Delete button	Item and its claims removed	Pass
21	Admin	Filter reports	Type = Lost	Only lost items listed	Pass
22	Session	Open /home without login	Direct URL	Redirect to login page	Pass
23	Session	User opens /admin	Logged in as student	HTTP 403 Access denied	Pass
24	Session	Logout	Click Logout	Session invalidated; /home needs login again	Pass
25	JDBC	Transaction on approval	Approve claim 2	Claim, item and other pending claims updated together (commit)	Pass
26	Security	SQL injection in login	`' OR '1'='1`	Login refused	Pass

TC	Module	Test case	Input	Expected result	Result
27	Security	Script in description	`<script>alert(1)</script>`	Shown as plain text, not run	Pass
28	Claim	Duplicate claim	Same user claims the same item again	Refused by ClaimServlet; UNIQUE key also blocks it in MySQL	Pass
29	Claim	Short proof (JavaScript off)	Proof "my calc" posted directly	Redirect with "needs more proof" message	Pass
30	Admin	Invalid status value	POST status=Hacked	HTTP 400 error page; status unchanged	Pass
31	Item	Future date (JavaScript off)	Date 2099-01-01 posted directly	Report refused; form shows an error	Pass

9.3 Database Verification

After test cases 13 and 17, the tables were checked with SQL queries:

```
mysql> SELECT item_id, item_name, status FROM items;
+---------+------------------------------+---------------+
| item_id | item_name                    | status        |
+---------+------------------------------+---------------+
|       1 | Black HP Laptop Bag          | Open          |
|       2 | College ID Card              | Claim Pending |
|       3 | Java Complete Reference Book | Open          |
|       4 | Samsung Galaxy Phone         | Closed        |
|       5 | Casio Scientific Calculator  | Open          |
|       6 | Set of Hostel Keys           | Open          |
+---------+------------------------------+---------------+

mysql> SELECT claim_id, item_id, claim_status, admin_remarks FROM claims;
+----------+---------+--------------+-----------------------------------------+
| claim_id | item_id | claim_status | admin_remarks                           |
+----------+---------+--------------+-----------------------------------------+
|        1 |       2 | Pending      | NULL                                    |
|        2 |       4 | Approved     | IMEI and wallpaper verified             |
|        3 |       6 | Rejected     | Room number does not match hostel records |
+----------+---------+--------------+-----------------------------------------+
```

10. Deployment

10.1 Deployment Environment

Item	Value
Server	Apache Tomcat 10.1 (Jakarta Servlet 6.0, JSP 3.1)
Context path	/CampusLostFound
URL	http://localhost:8080/CampusLostFound/
Database	MySQL 8 on localhost:3306, database campus_lost_found
JDBC driver	WEB-INF/lib/mysql-connector-j-8.4.0.jar

10.2 Steps to Deploy in Eclipse

1. Run `database/campus_lost_found.sql` in MySQL Workbench. It creates the database, tables, stored procedure and sample data.
2. Set the MySQL user name and password in `DBConnection.java`.
3. In Eclipse choose File → Import → Existing Projects into Workspace and select the `CampusLostFound` folder.
4. Add the server runtime: Window → Preferences → Server → Runtime Environments → Add → Apache Tomcat v10.1.
5. Right-click the project → Properties → Targeted Runtimes → tick Apache Tomcat v10.1.
6. Right-click the project → Run As → Run on Server.
7. Open http://localhost:8080/CampusLostFound/ in the browser.

10.3 Deploying as a WAR File

1. Right-click the project → Export → WAR file (or run `mvn package` in the Maven version).
2. Copy `CampusLostFound.war` into the `webapps` folder of Tomcat.
3. Start Tomcat using `bin/startup.bat` (Windows) or `bin/startup.sh` (Linux).
4. Tomcat unpacks the WAR and the application becomes available at /CampusLostFound.

10.4 Default Accounts

Role	Email	Password
Administrator	admin@campus.edu	admin123
Sample student	student@campus.edu	user123

11. Results and Expected Output

11.1 Expected Output Achieved

#	Expected output	Achieved by	Status
1	User registration and login	RegisterServlet, LoginServlet (Fig. 8.1, 8.2)	Achieved
2	Reporting lost and found items	ReportItemServlet (Fig. 8.4, 8.5)	Achieved
3	Searching for reported belongings	SearchServlet (Fig. 8.6)	Achieved
4	Viewing item details	ItemDetailsServlet (Fig. 8.7)	Achieved
5	Submitting ownership claims	ClaimServlet (Fig. 8.7, 8.9)	Achieved
6	Administrator verification of claims	AdminServlet → ClaimDAO.decideClaim() (Fig. 8.11, 8.12)	Achieved
7	Updating item recovery status	Status drop-down and auto-close (Fig. 8.13, 8.14)	Achieved
8	Database operations using JDBC	INSERT, SELECT, UPDATE, DELETE, CallableStatement, transactions	Achieved
9	Session management	HttpSession, cookie, URL rewriting, 403, logout (Fig. 8.15, 8.16)	Achieved
10	Integration of Servlets, JSP and JavaBeans using MVC	Servlet → DAO/Bean → JSP for every page	Achieved

11.2 Course Outcome Mapping

Course Outcome	Contribution of this assignment
CO1	HTML forms with validation; GET and POST request-response communication
CO2	Servlets for item reporting, searching and session management
CO3	Dynamic JSP pages using directives, scripting elements, actions and implicit objects
CO4	Integration of JavaBeans, Servlets and JSP in the MVC architecture
CO5	JDBC connectivity, SQL CRUD operations, stored procedure and transaction management
CO6	A complete database-driven web application using Advanced Java technologies

12. Conclusion and Future Enhancements

12.1 Conclusion

The Campus Lost-and-Found Management Portal was designed and developed successfully using Advanced Java technologies. The application gives students, faculty and visitors a single place to report lost and found belongings, search them and claim them with proof, while the administrator verifies every claim before an item is handed over.

The project applies every topic of the Advanced Java Programming syllabus in one working application: HTML forms and HTTP methods, the Servlet life cycle and ServletConfig, JSP directives, scripting elements, standard actions and implicit objects, JavaBeans, the MVC architecture, JDBC with Statement, PreparedStatement, CallableStatement, ResultSet and transactions, and session management with HttpSession, cookies and URL rewriting. All functional requirements were met and all test cases passed.

12.2 Future Enhancements

- Upload photographs of items to make identification easier.
- Send e-mail or SMS alerts when a matching found item is reported.
- Automatic matching of lost and found reports using keywords and categories.
- QR-code stickers for ID cards and laptops that link to the owner's profile.
- Password reset using OTP, and a connection pool (JNDI DataSource) for better performance.
- A mobile application that uses the same database through REST services.

References

1. Uttam K. Roy, *Web Technologies*, Oxford University Press, 1st Edition, 2011.
2. Marty Hall and Larry Brown, *Core Servlets and JavaServer Pages: Core Technologies*, 2nd Edition, Pearson Education, 2003.
3. Mike Murach and Michael Urban, *Murach's Java Servlets and JSP*, 3rd Edition, Mike Murach & Associates, 2014.
4. Paul Deitel and Harvey Deitel, *Java: How to Program*, 10th Edition, Pearson Education, 2015.
5. Herbert Schildt, *Java: The Complete Reference*, 12th Edition, McGraw Hill, 2022.
6. Jason Hunter, *Java Servlet Programming*, 2nd Edition, O'Reilly Media, 2001.
7. Hans Bergsten, *JavaServer Pages*, 3rd Edition, O'Reilly Media, 2003.
8. Jakarta Servlet 6.0 and Jakarta Server Pages 3.1 Specifications, Eclipse Foundation.
9. MySQL 8.0 Reference Manual, Oracle Corporation.